

# Plant Disease Recognition

## Abstract

In this project, we developed a classification pipeline for plant disease recognition of healthy, rusty, and powdery classes of apple leaves. Our approach integrates six feature families—edge detection, color channel analysis, texture descriptors, blob detection, luminance histograms, and ResNet. We combined a structured preprocessing pipeline and multiple machine-learning models to evaluate classification performance across the three classes. We trained and compared several variations of our models, including logistic regression, support vector machines, random forests. By evaluating accuracy, robustness, and generalization, we found the random forest classifier performed best.

## 1 Introduction

This project uses a Kaggle data-set of apple leaf images that were web-scraped and publicly available. The dataset contains 1,530 images across three classes—Healthy (528), Powdery (500), and Rust (504)—each split into training (430–458 images), validation (20 images), and test sets (50 images). Image file sizes range from approximately 200 KB to 1 MB.

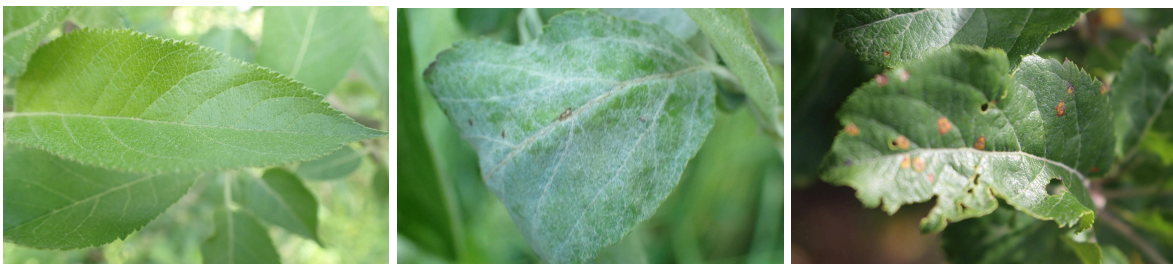


Figure 1.1: Example images for the healthy class (left), powdery class (middle), and rust class (right).

Most of the images in our dataset had a resolution of  $4000 \times 2672$  pixels, with a few outliers at  $5184 \times 3458$  and  $2592 \times 1728$  pixels. To standardize the dataset, we first removed backgrounds using the RemBG preprocessing tool, then applied center-cropping to achieve a square 1:1 aspect ratio. All images were subsequently normalized to a fixed resolution of  $256 \times 256$  pixels and

converted to the uint8 format to ensure consistency and compatibility across the dataset. Here's a sample Mean and Standard Deviation of Pixel Intensities plot in Figure 1.2 from the resulting data standardization.

We applied a wide range of feature-extraction techniques, beginning with simple descriptors such as color channels and luminance histograms, and extending to more advanced methods including edge-based, blob-based, and texture-based analyses. To evaluate model performance, we trained several classifiers, including logistic regression, support vector machines, random forests, and a ResNet-based deep model used

for high-level feature representation. Our baseline untuned Logistic Regression model without feature extraction achieved approximately 46% accuracy, highlighting the difficulty of predicting the classes with minimal preprocessing. By incorporating ResNet embeddings and training a Random Forest classifier on these features, we improved performance to 96% accuracy on the test set, demonstrating the value of combining traditional feature engineering with deep learned representations for robust plant disease classification.

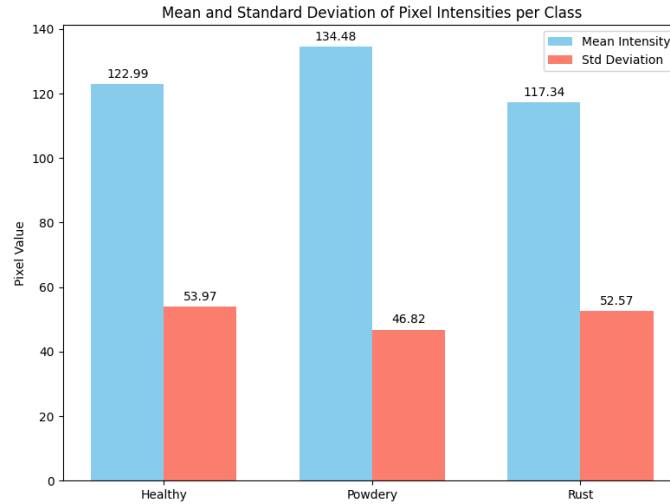


Figure 1.2: Mean and Standard Deviation of Pixel Intensities per Class

## 2 Feature Extraction

In this section, we discuss the features we considered for inclusion in our classification. We discuss the motivation and the findings for feature selection.

### 2.1 Edge Detection

Edge detection plays a key role in our pipeline because it highlights object boundaries, emphasizes structural outlines, and reduces visual complexity. Across our dataset, powdery-mildew leaves tend to spike at 255 due to the mildew's highly reflective surface, rusty leaves show rapid pixel transitions between healthy and infected regions, and healthy leaves maintain uniform color with solid, well-defined edges.

The image below shows a Canny applied rusty leaf image (Figure 2.1.1) and the edge density plots across the rows and columns (Figure 2.1.2).

We applied the OpenCV Canny edge detection model with a sensitive threshold range between 50 and 150 to capture subtle transitions. This method uses a Gaussian filter to reduce noise sensitivity then computes for edge strength. However, the resulting binary edge maps are too high-dimensional (as seen in Figure 2.1.2 tSNE) to combine effectively with other features for models like ResNet or logistic regression, so we simplified them into edge-pixel counts. The histogram of these counts is skewed to the right, indicating that the algorithm is detecting fewer edges than expected. This suggests potential improvements such as contrast enhancement or reducing the Gaussian kernel size to avoid blurring the fine boundaries that distinguish rusty from powdery leaves.

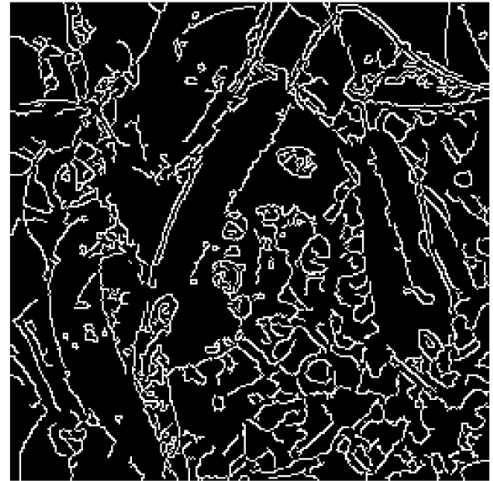


Figure 2.1.1 Canny Edge Detection on Rusty Leaf

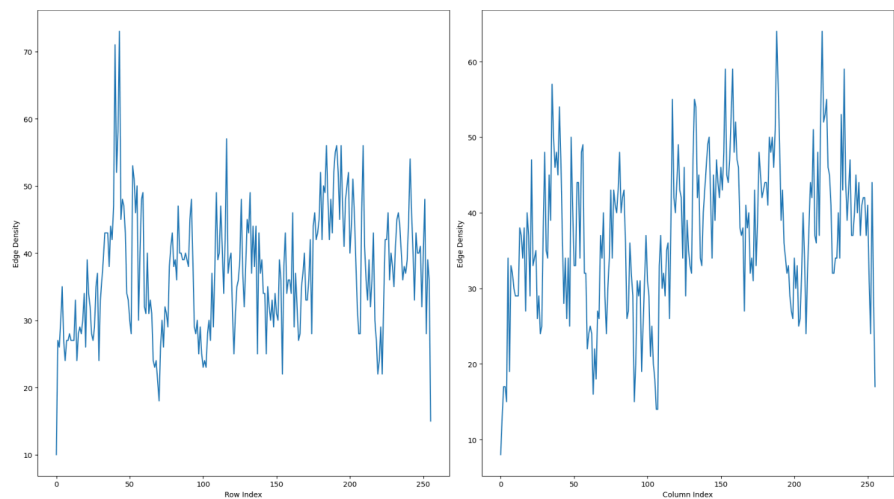


Figure 2.1.2 Edge Density Plot per Rows and Column

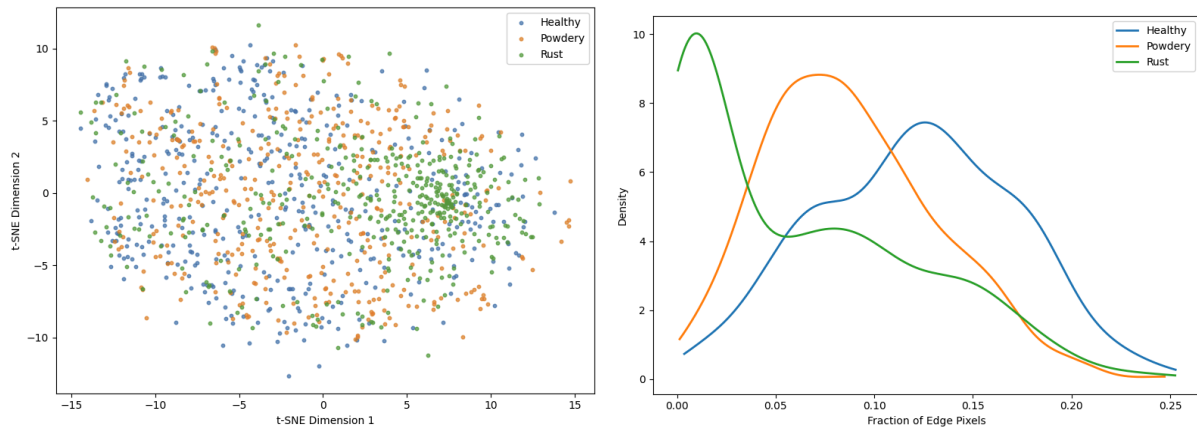


Figure 2.1.2: tSNE (left) and Smoothed Histogram (right) per class

## 2.2 Blob Detection

To pursue blob detection, we propose two easy-to-understand, shape-based features, blob count and blob area, which quantify the number and overall area of localized points of interest in an image, respectively. Our motivation for pursuing the blob detection features was mostly intuitive—we aimed to detect spots or lesions present within the rust and powdery classes. Blob count captures how frequently these points of interest occur, whereas blob area captures the total affected leaf surface. Blob count and blob area together represent biologically driven, scalar features which should be useful for our classifiers and distinct from other, more typical features.

In application, we utilized the OpenCV function SimpleBlobDetector, which was applied to grayscale versions of the plant images with their backgrounds removed. Blobs were detected as light spots, i.e., large pixel magnitude, and we allowed for the detection of blobs within a range of 1-5000 pixel area. Instead of a typical conversion of RGB to grayscale, we found our best results when interpreting the red channel, or first color channel, as grayscale pixel magnitudes. We



Figure 2.2.1: Blob detection applied to an example of the rust class

believe that the effectiveness of this approach was driven by the color of the lesions for the rust class, which were predominantly red, orange, and yellow.

As shown in figure 2.2.1, the blob detection algorithm also captures blobs created by reflections or indentations on the leaves. Though we experimented with a thresholding approach to target specifically red blobs, we observed the best classification results when no thresholds were used. The blob area feature was intended to help distinguish between cases with small, lighting induced blobs versus larger lesions. However, the absence of a thresholding approach led to the somewhat unintended results of the healthy class typically having the largest blob counts and blob areas, followed by the powdery class (figure 2.2.2). Regardless, the resultant features show a clear separation between classes when comparing smoothed histograms, as shown in figure 2.2.2, and these features yielded a promising 57% validation accuracy by themselves when tested with a logistic regression classifier.

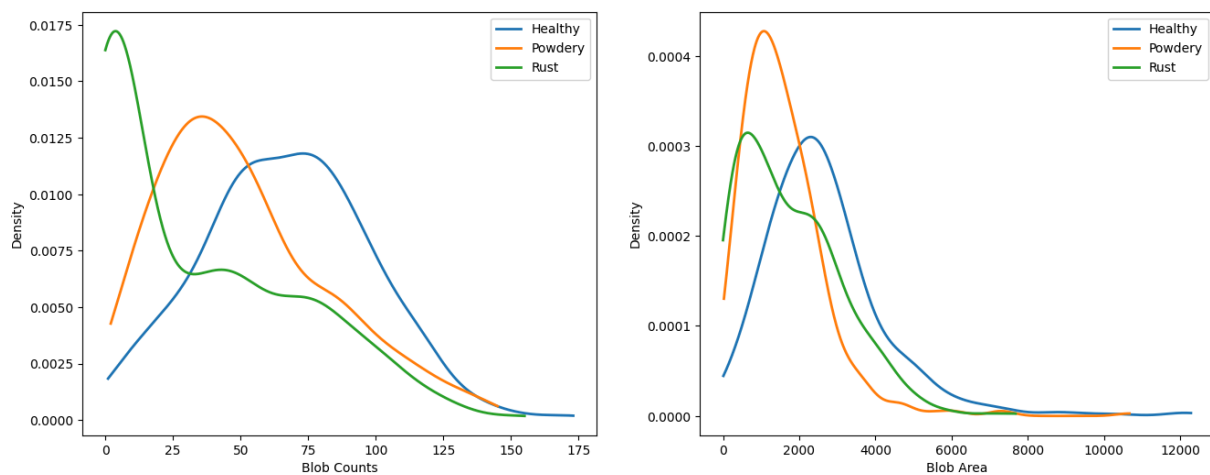


Figure 2.2.2: Smoothed histograms of blob counts (left) and blob areas (right) per class

## 2.3 Color

Plant diseases manifest visually through leaf discoloration, as the once green leaf begins to turn a brownish-red. To capture the visual differences among the discoloration of the diseased leaves in each class, we implemented color histogram analysis to capture the disease-induced changes to the color of each leaf. Our goal was to extract features from the color of each image and prepare features that can be used to train a model as well as be used in conjunction with the ResNet50 deep learning CNN.

This was achieved by plotting the abundance of each red, green, and blue channel at each intensity generating histograms with 256 bins spanning the full intensity range. From these plots, features were extracted by capturing statistical metrics from each of the three distributions for every image. Six measures were applied to all three color channels, resulting in 18 standalone features:

- mean
- standard deviation
- skewness
- median
- entropy
- peak count

In addition to these basic features, we also extracted cross-channel comparative features, including red-to-green and blue-to-green mean ratio, which aims to capture the change in the color of the leaf as disease progresses.

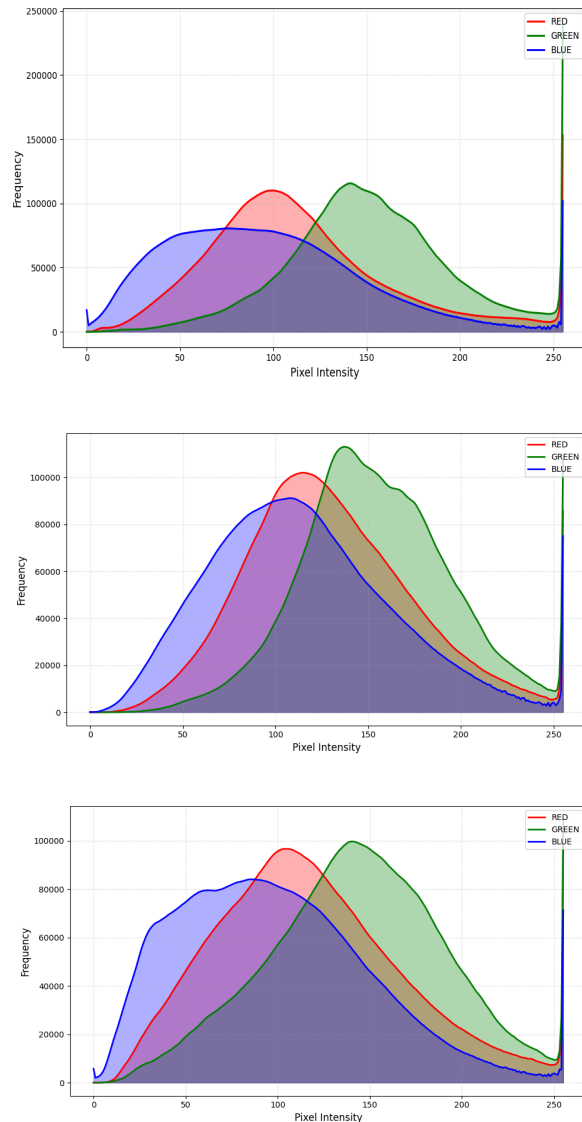


Figure 2.3.1: Histograms of average pixel intensity and frequency for each color channel for each class

After these features were extracted, a random forest model was trained and Gini importance was performed with the purpose of qualifying each feature for inclusion in the final classification model. Among the results, the ratio of the red and green color channel means and the mean of the green channel stood out as the two strongest predictors. Due to all extracted features having a gini importance test value of above .02 coupled with quick training time for all models, all features were deemed significant and included in the final model.

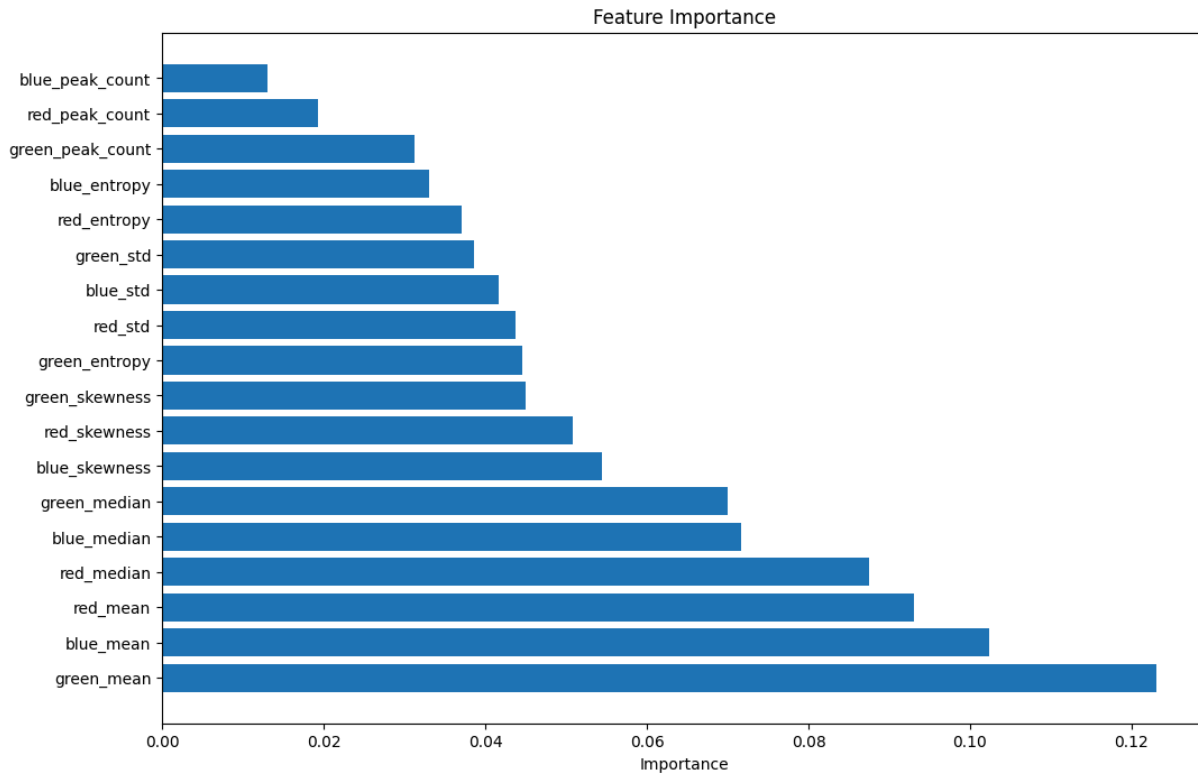


Figure 2.3.2: Gini Feature Importance for Random Forest

## 2.4 Texture Detection

Texture can play a major role in distinguishing the three leaf categories because diseases change both the surface structure and the consistency of gradients across the leaf. Unlike features based on color, texture responds to how “rough”, “smooth”, or “clustered” the leaf surface is, which is especially important for *Powdery* and *Rust* classes. In other words, powdery mildew produces a fine, dense, grain-like texture across large areas of the leaf, while rust forms circular pustules with strong gradient transitions. In contrast, healthy leaves have smoother and more uniform patterns with defined and uniform venation lines. These differences can make texture a very informative feature.

To capture these differences, Histogram of Oriented Gradients (HOG) was used. HOG converts each leaf into grayscale, divides it into small spatial cells, and computes gradient orientation histograms that summarize how edges change direction across the image. The resulting feature vector captures the surface unique patterns of each leaf.

For this dataset, HOG was expected to be effective because rust and powdery leaves signatures were orientation heavy:

- *Powdery* - many small and local gradient
- *Rust* - strong circular edges and sharp transitions
- *Healthy* - smooth global gradients with fewer sharp changes

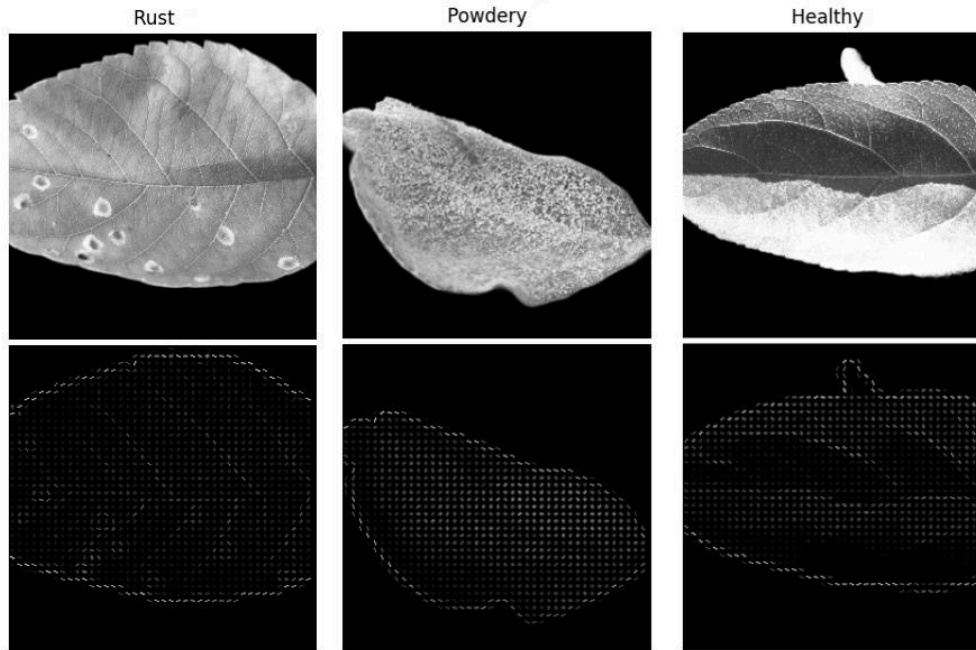


Figure 2.4.1: HOG visualization of each class

To improve efficiency, PCA was applied to reduce dimensionality (e.g., (1322, 6912)  $\rightarrow$  (1322, 100)) before these vectors were passed into models. Therefore, the t-SNE visualisation (Figure 2.4.2) showed some degree of class clustering, but the separation was weaker than expected in the preliminary tests.

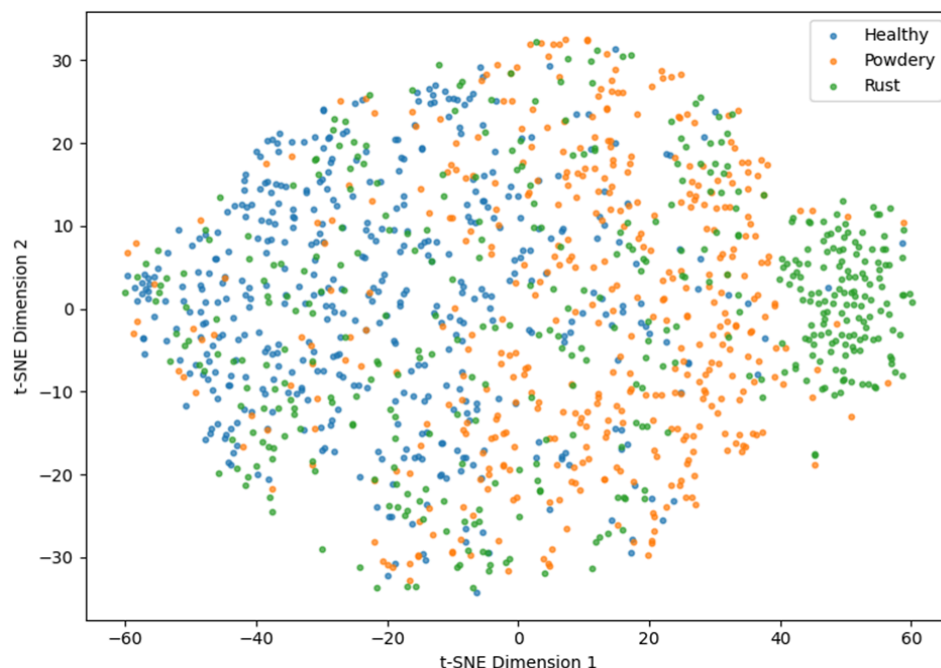


Figure 2.4.2: t-SNE visualization of the HOG texture detection feature

## 2.5 Luminance Histogram

A luminance histogram is an intensity based feature that describes how pixel brightness values are distributed across an image. By converting each plant image to grayscale and counting how often each intensity level occurs, this feature aims to capture overall lighting patterns and contrast of images, regardless of spatiality. Our inspiration for adding this feature was in observing how the luminance distributions can differ between the healthy, powdery, and rust classes—e.g., healthy leaves tend to be more

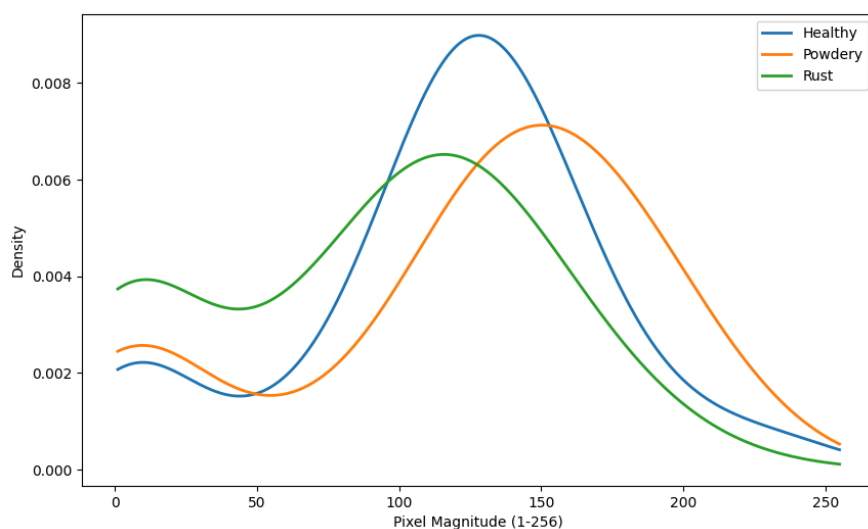


Figure 2.5.1: Smoothed histograms of luminance histograms (grayscale pixel magnitude distribution) per class

reflective/glossy, powdery leaves tend to be whiter and more uniform in brightness, and rust leaves tend to be darker.

To generate this feature, we converted the images—with their backgrounds removed—to grayscale and counted the occurrence of pixel magnitudes 1-255. Notably, 0 is excluded here because of the black space generated by the background removal process, which would skew the histograms and make less emphasis on the leaves themselves. Additionally, because the images were preprocessed to be uint8 datatype, binning was straightforward with 255 bins. Figure 2.5.1 shows clear separation between classes and results which agree with our theoretical expectations—the powdery class tends to be the brightest and the rust class tends to be the darkest. The t-SNE plot in figure 2.5.2 shows clear clustering between classes, with the tightest clustering being evident for the rust class. As a preliminary test, a logistic regression classifier trained on this feature yielded 60% accuracy for the validation set.

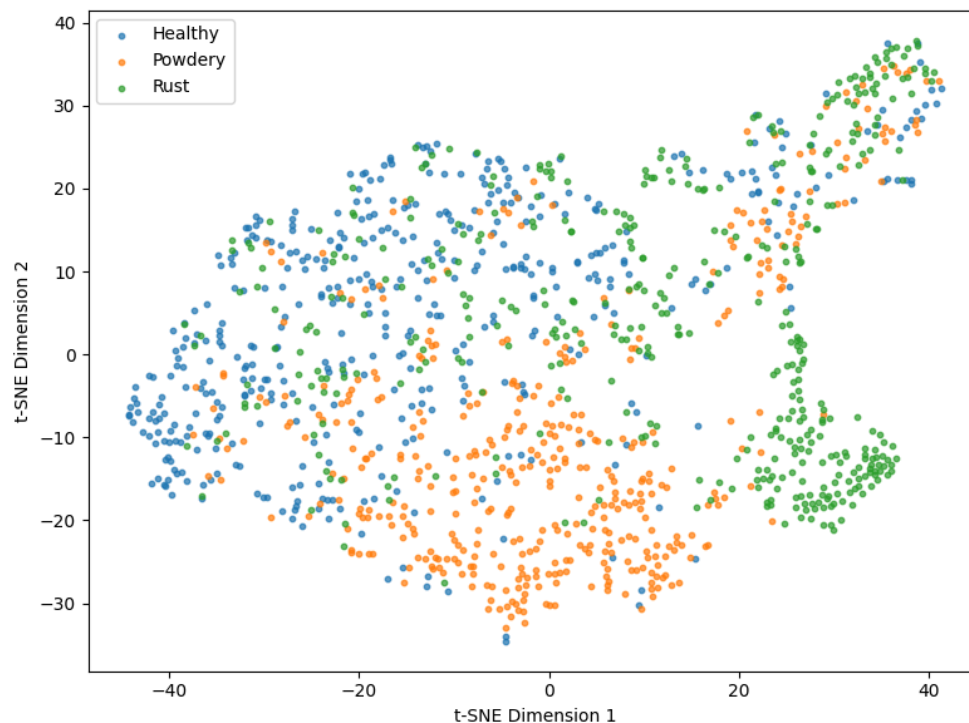


Figure 2.5.2: t-SNE visualization of the luminance histogram feature per class

## 2.6 ResNet50

ResNet50 is a deep learning convolutional neural network (CNN) with 50 layers that was originally trained on the ImageNet dataset, which contains millions of labeled images for thousands of objects. Even among other deep learning neural networks, the ResNet architecture

was distinguished for its use of residual learning when it was published in 2015<sup>1</sup>. As a pretrained CNN, ResNet50 offers a robust and powerful feature vector for many image classification problems. Though we utilized a variety of features for this classification project, the addition of a feature vector derived from ResNet50 allows for the capability to detect features that the human eye might not notice.

To create a feature vector using ResNet50, the CNN was imported pretrained from the tensorflow library. ResNet was trained on images of size 224 by 224, so it was necessary to resize our images. Further, the classification head was removed from ResNet50 and the output from the last dense layer of the neural network was imported directly as the feature vector, resulting in a vector of size 2048. We experimented with providing ResNet50 our processed images with the backgrounds removed, however, we observed improved results when using the images with their backgrounds still present. As shown in figure 2.6.1, a t-SNE visualization of the resultant feature displays striking separation and clustering between classes.

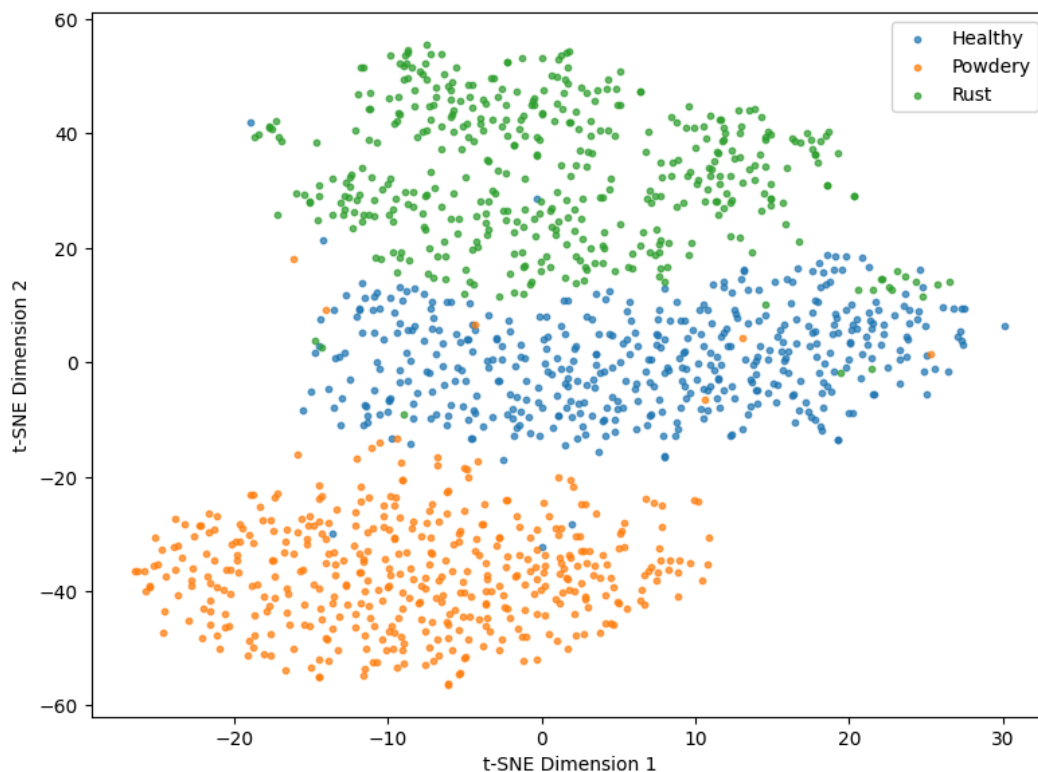


Figure 2.6.1: t-SNE visualization of the ResNet50 feature

---

<sup>1</sup> 1) He et. al, "Deep Residual Learning for Image Recognition", 2015, <https://arxiv.org/abs/1512.03385>

## 3 Classification

In this section, we describe the classification models we applied for our project, as well as the results of these models.

### 3.11 Baseline

The use of a baseline model is important for contextualizing the benefit and progress of using any machine learning model. For multiclass classification, a common baseline model is a majority class classifier, however our plant image dataset contains approximately balanced classes. As an alternative, we built a baseline model by converting the images to grayscale and flattening them into feature vectors, using which a logistic regression model was trained and evaluated. This is a simplistic approach for training machine learning classifiers, and our expectation for our baseline was relatively poor performance. As shown in figure 3.4.1, the resultant validation accuracy was 43%. This accuracy is already lower than the accuracy of some of our preliminary models which were trained on single features, such as the blob detection (57%) and luminance histogram (60%) features.

### 3.12 Logistic Regression

Logistic Regression was chosen as one of our classification models because it works well with high-level processed features as opposed to raw data and is relatively quick to train and tune hyperparameters. Logistic Regression learns linear combinations from features which differ from the logic of the support vector classifier and random forest's decision trees, differentiating it from the other machine learning classification models used.

As expected, the our Logistic Regression model ran relatively fast, completing training in 69 seconds for the ResNet model and only 13 seconds for the standalone model. In total, four logistic regression models were trained, the logistic regression classifier's performance with tuned, untuned, ResNet, and no ResNet inclusion. The following are the results of the test accuracy for each of the four trained logistic regression classifier models:

*Un-tuned, no ResNet: 0.73*

*Un-tuned, ResNet: 0.81*

*Tuned, no Resnet: 0.73*

*Tuned, Resnet: 0.83*

One key takeaway from the Logistic Regression model is that the tuned ResNet model exhibited a lower precision compared to recall( $\Delta = 0.09$ ) for the healthy class, and a higher precision compared to recall( $\Delta = 0.07, 0.03$ ) for the powdery and rust classes respectively. This model therefore liberally predicts healthy and will occasionally misclassify diseased leaves as healthy.

Across a range of [0.01, 0.1, 1, 10], we additionally identified hyperparameter C=0.01 as optimal with a cross-validation accuracy of 87.21% averaged across 5 folds on the training set.

|                                               |           |        |          |         |  |
|-----------------------------------------------|-----------|--------|----------|---------|--|
| LR Tuned Validation Results:                  |           |        |          |         |  |
|                                               | precision | recall | f1-score | support |  |
| 0                                             | 0.74      | 0.85   | 0.79     | 20      |  |
| 1                                             | 0.95      | 0.90   | 0.92     | 20      |  |
| 2                                             | 0.89      | 0.80   | 0.84     | 20      |  |
| accuracy                                      |           |        | 0.85     | 60      |  |
| macro avg                                     | 0.86      | 0.85   | 0.85     | 60      |  |
| weighted avg                                  | 0.86      | 0.85   | 0.85     | 60      |  |
| Test Results:                                 |           |        |          |         |  |
|                                               | precision | recall | f1-score | support |  |
| 0                                             | 0.77      | 0.86   | 0.81     | 50      |  |
| 1                                             | 0.89      | 0.82   | 0.85     | 50      |  |
| 2                                             | 0.85      | 0.82   | 0.84     | 50      |  |
| accuracy                                      |           |        | 0.83     | 150     |  |
| macro avg                                     | 0.84      | 0.83   | 0.83     | 150     |  |
| weighted avg                                  | 0.84      | 0.83   | 0.83     | 150     |  |
| Average AUC (macro) on validation set: 0.9767 |           |        |          |         |  |
| Average AUC (macro) on test set: 0.9586       |           |        |          |         |  |
| CPU times: total: 49.7 s                      |           |        |          |         |  |
| Wall time: 1min 9s                            |           |        |          |         |  |

Figure 3.12.1: Tuned Logistic Regression Classifier With ResNet Validation and Test Results

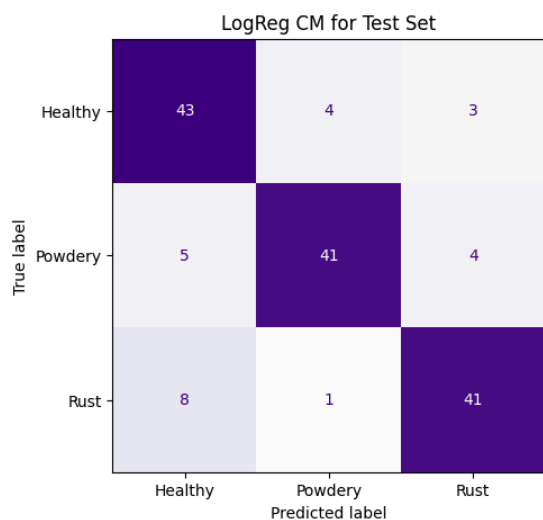


Figure 3.12.2: Confusion Matrix for Logistic Regression with ResNet

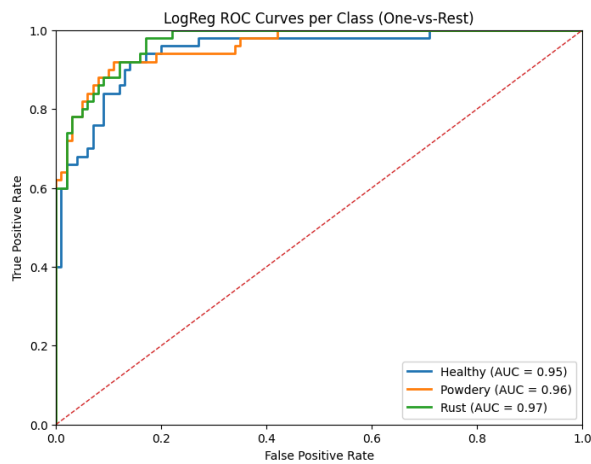


Figure 3.12.3: ROC Curve for Logistic Regression with ResNet

### 3.13 Support Vector Classifier

SVC was selected as one of the classifiers because it handles high-dimensional features extremely well, especially after PCA application. Many of the features, including HOG, luminance histograms, and color statistics, produce nonlinear relationships between classes, and the kernel-based approach of SVC works very well for capturing these boundaries.

During the test run, SVC achieved generalization performance comparable to the other classical models, indicating that it was able to learn meaningful separation of the classes regardless of the feature representation. However, the main drawback of SVC in the pipeline was runtime efficiency. Training SVC, especially on large or high-dimensional feature sets such as HOG was noticeably slower than other models. For hyperparameter tuning of our SVC classifier, we searched across the normalization constant C, the gamma, and the kernel type, and found an optimal configuration of {'C': 100, 'gamma': 'scale', 'kernel': 'rbf'}. SVC performed on par with other models in terms of accuracy, but its computational runtime gap became even wider compared to lighter models that delivered similar results in less time (Figure 3.13.1).

```
SVC Tuned
Validation Results:
      precision    recall  f1-score   support

     0       0.75      0.90      0.82        20
     1       0.95      0.95      0.95        20
     2       0.94      0.75      0.83        20

 accuracy          0.87        60
 macro avg         0.88      0.87      0.87        60
 weighted avg      0.88      0.87      0.87        60

Test Results:
      precision    recall  f1-score   support

     0       0.80      0.82      0.81        50
     1       0.80      0.82      0.81        50
     2       0.79      0.76      0.78        50

 accuracy          0.80       150
 macro avg         0.80      0.80      0.80       150
 weighted avg      0.80      0.80      0.80       150
...
Average AUC (macro) on validation set: 0.9838
Average AUC (macro) on test set: 0.9477
CPU times: total: 22.1 s
Wall time: 3min 22s
```

Figure 3.13.1: Tuned SVC Classifier With ResNet Validation and Test Results

Overall, SVC served as a useful classifier showing almost identical accuracy for all three classes as represented by the confusion matrix (Figure 3.13.2) and ROC curves plot (Figure 3.13.3), but its runtime overhead made it less favorable for the final optimized pipeline.

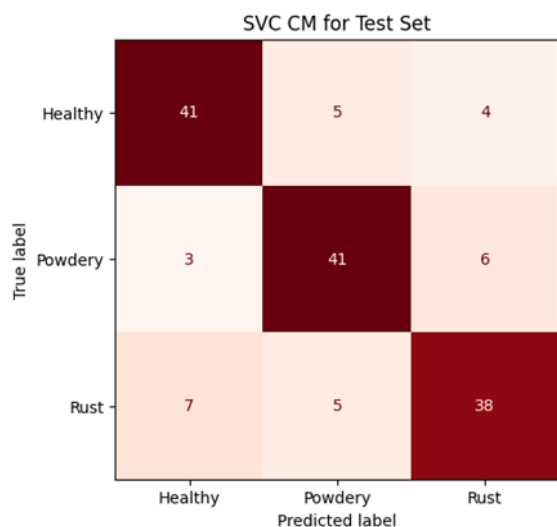


Figure 3.13.2: Confusion Matrix for SVC with ResNet

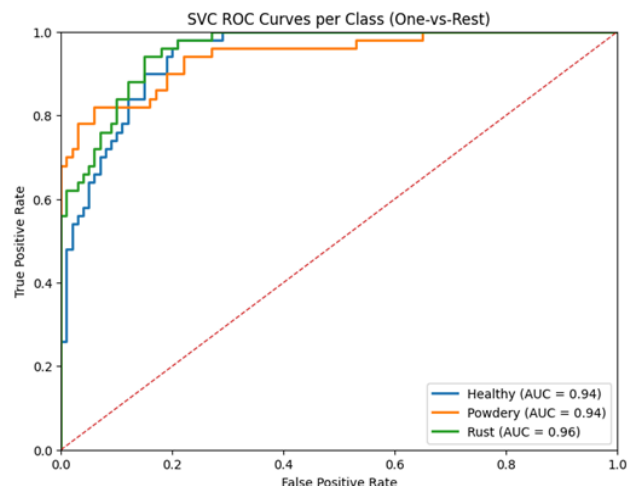


Figure 3.13.3 ROC Curve for SVC with ResNet

### 3.14 Random Forest

Random forest was a strong fit for this project as it handles non-linear feature interactions extremely well by allowing the decision trees to naturally pick up patterns in color, texture, blobs, and edge features without much hands-on tuning. This ensemble structure reduces overfit and noise.

We ran the Random Forest classifier with ResNet and without ResNet. The Random Forest with ResNet model outperformed all other classifiers with near-perfect validation accuracy of 1.0 and strong generalization results of 0.96 while maintaining a fast runtime of 40 seconds. It also achieved the highest AUC score of 1.0 on validation and 0.99 on test data. This shows the classifier can reliably rank the correct class and catches true-positives 99% of the time, as seen in Figure 3.14.1. The only down-side of the results is the Powdery leaves have a 0.99 AUC score (shown in Figure 3.14.2), We can see that the misses result from the confusion between Powdery and Healthy leaves. For hyperparameter tuning, we searched across number of estimators, maximum depth, minimum samples split, and minimum samples leaf parameters, and found an

optimal configuration with {'max\_depth': None, 'min\_samples\_leaf': 1, 'min\_samples\_split': 5, 'n\_estimators': 200}.

Overall, the random forest model with ResNet features performed the best in terms of accuracy and runtime, marking it as the clear choice for application in automation of plant disease detection. We hypothesize that the decision tree architecture of random forest excelled at identifying highly important features that were not specific to the training split, which contributed to its role as the best model of the three.

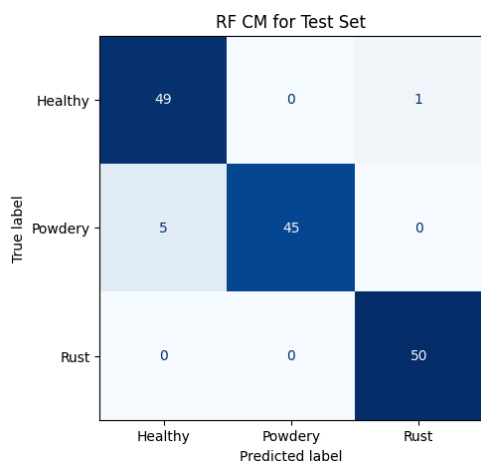


Figure 3.14.1: Confusion Matrix for Random Forest with ResNet

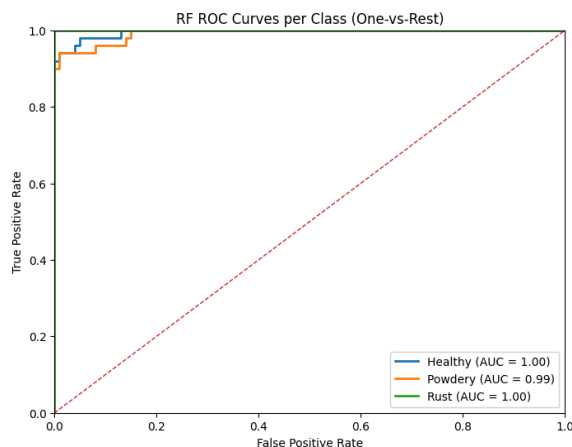


Figure 3.12.2: ROC Curve for Random Forest with ResNet

## 3.2 Generalizability

The final three tuned models with ResNet50 features perform well and demonstrate good generalizability, maintaining consistent performance from training through validation and to testing. The following is a breakdown of the training, validation, and testing accuracy for the baseline model as well as the three fully tuned and trained models with and without ResNet50 features.

Baseline Support Vector Classifier - Severe Overfitting. This SVC model trained on flattened grey-scaled images exhibits 57-54 percentage point drop from training to validation and test accuracy respectively. drop from training to validation accuracy, indicating the model memorized the raw features without learning generalized patterns. The following indicates the accuracy for the 3 splits in the format *train / validation / test*.

*Baseline: 1.0 / 0.43 / 0.46*

Support Vector Classifier - Stable Generalization. The SVC model demonstrates tightly contained accuracy values across all splits indicating a highly stable and generalizable model when ResNet50 features are excluded. There is however a slight accuracy drop from validation to test accuracy for the model including ResNet features as there is a 0.07 drop in accuracy from validation to test. This indicates that SVCs behavior may capture more training-specific patterns in the data, but still generalizes within an acceptable falloff.

*SVC w/o ResNet: 0.76 / 0.78 / 0.76*

*SVC w/ ResNet: 0.86 / 0.87 / 0.80*

Logistic Regression Classifier - Best Generalization. Logistic Regression demonstrates the most stable generalization, with only a 0.04 difference in accuracy from training over to test. The tuned model with and without shows a gradual and small decline in accuracy, resulting in the model being highly generalizable.

*LogReg w/o ResNet: 0.77 / 0.75 / 0.73*

*LogReg w/ ResNet: 0.87 / 0.85 / 0.83*

Random Forest - Good Generalization, Highest Accuracy. The Random Forest classifier maintains an increase in accuracy from training to validation for the model without ResNet50 features, and a larger difference between validation and test accuracy. The inclusion of ResNet50 features however drastically improve generalization stability, with training, validation, and test accuracy staying within 4% of each other.

*RF w/o ResNet: 0.78 / 0.83 / 0.74*

*RF w/ ResNet: 0.98 / 1.0 / 0.96*

All three models demonstrate acceptable generalization stability. Random forest in particular experienced the greatest improvement to generalization as a result of the inclusion of ResNet features. Overall, the random forest classifier demonstrates the highest absolute performance with good generalization, making it the clear winner for deployment on additional samples.

### **3.4 Efficiency vs. Accuracy**

The tests were performed to compare accuracy and runtime among various setups using three classification models: Logistic Regression (LogReg), Support Vector Classification (SVC), and Random Forest (RF). All models were executed with and without ResNet features.

Train/Validation/Test Accuracy, AUC on the Validation and Test sets, and Total Runtime were measured and analyzed for all setups (Figure 3.4.1).

Without ResNet features, LogReg performed relatively well, with accuracy and AUC measurements of 0.77 / 0.75 / 0.73 and 0.94 / 0.89, respectively, taking 13.9 seconds. However, with ResNet features, accuracy and AUC values were 0.87 / 0.85 / 0.83 and 0.98 / 0.96, respectively, at a cost of 1 minute 9 seconds. It clearly represents a trade-off situation, as the ResNet model performs better at the cost of significantly larger training times.

Resulting Accuracy and AUC values for SVC were 0.76 / 0.78 / 0.76 and 0.94 / 0.90, respectively, with a runtime of 1 minute 16 seconds. When SVC included ResNet features, accuracy slightly increased to 0.86 / 0.87 / 0.80, and AUC values reached 0.98 / 0.95, but with the longest runtime among all model setups - 3 minutes 22 seconds. A very good trade-off for efficiency exists here, as SVC with ResNet is more precise but at a relatively higher computational cost.

RF showed accuracy and AUC of 0.78 / 0.83 / 0.74 and 0.96 / 0.92 without ResNet feature sets within 18.6 seconds. RF with ResNet feature sets obtained the maximum accuracy among all models with 0.98 / 1.0 / 0.96 and maximum AUC values of 1.0 / 0.99 within a runtime of 39.4 seconds. As a result, the RF model improved by ResNet shows the greatest enhancement.

| Model             | Accuracy<br>(Train/Val/Test) | AUC<br>(Val/Test) | Runtime  |
|-------------------|------------------------------|-------------------|----------|
| Baseline LogReg   | 1.0 / 0.43 / 0.46            | 0.66 / 0.66       | 15.3 s   |
| SVC w/o ResNet    | 0.76 / 0.78 / 0.76           | 0.94 / 0.90       | 1min 16s |
| SVC w/ ResNet     | 0.86 / 0.87 / 0.80           | 0.98 / 0.95       | 3min 22s |
| LogReg w/o ResNet | 0.77 / 0.75 / 0.73           | 0.94 / 0.89       | 13.9 s   |
| LogReg w/ ResNet  | 0.87 / 0.85 / 0.83           | 0.98 / 0.96       | 1min 9s  |
| RF w/o ResNet     | 0.78 / 0.83 / 0.74           | 0.96 / 0.92       | 18.6 s   |
| RF w/ ResNet      | 0.98 / 1.0 / 0.96            | 1.0 / 0.99        | 39.4 s   |

Figure 3.4.1: Summary of classifier results

In terms of accuracy alone, the optimal configuration would be RF with ResNet features that showed the highest accuracy and AUC value on all datasets. On optimizing for efficiency, LogReg without ResNet features is the quickest with a runtime of 13.9 seconds, but it has lower accuracy compared to the rest. The trade-off here clearly brought out by these experiments is that ResNet feature inclusion will always result in better accuracy but will be significantly slower, and vice versa.

## 4 Conclusion

Through our process of background removal, feature extraction, addition of ResNet50 features, and comparison of three separate classification algorithms, we believe we have succeeded in producing a highly accurate and generalizable model capable of identifying three categories of disease among apple leaves. The random forest model with ResNet features emerged as the optimal model due to its high accuracy, low runtime, and stable generalizability marked by the small difference in accuracy between train, validation, and test splits. Our process additionally brought forth several key takeaways:

1. Inclusion of computer vision techniques and simple and complex features dramatically improved the model compared to the baseline. This resulted in an overall improvement in test accuracy from 0.46 to 0.96 for the baseline logistic regression classifier compared to our final tuned random forest model with ResNet features. The addition of ResNet50 features significantly increased the performance of all models, particularly random forest, resulting in an accuracy increase of 0.22 when comparing the final RF model to the version omitting ResNet50 features.
2. The addition of the extracted features did not significantly increase the runtime of the model. While ResNet50 features yielded the most significant increase to training time, the highest performing random forest model with Resnet features only took 2.57 times as long to train.
3. Different models exhibited different predication biases. Logistic regression showed overclassification of healthy samples and underclassification of diseased samples, while SVC exhibited the most balanced precision-recall ratio.
4. This research highlights areas of further exploration. While the Random Forest algorithm excelled for our data, the SVC and Logistic Regression models responded to simple and complex features differently while also demonstrating decent ability to classify and generalize. Future work should explore other plants and disease types while continuing to incorporate various simple and complex features.

In addition to developing a highly accurate model, our work showcases the efficacy of a hybrid approach of combining processed and grounded features with deep learning representations of the raw image data, demonstrating the value of computer vision techniques in image

classification. With 96% test accuracy, 0.99 AUC, and the lowest runtime for all ResNet models with a training time of only 39.4 seconds, our tuned random forest model with ResNet demonstrates production ready performance and ability to automate plant disease monitoring quickly and effectively.

# Contributions

- Bonnie Yang
  - Contributed to exploratory data analysis for the overall data-set, an extensive deep dive into edge detection using Sobel and Canny, logistic regression/PCA, and associated visualizations (tSNE, ROC, CM). Integrated this code into the final Collab Jupyter Notebook along with the EDA, Feature Extraction, Results, and Visualization sections. Ran all the classifiers to help compare and contrast model results. For the final report, I contributed to the Abstract, Introduction, Edge Detection, and Random Forest.
- Konstantin Khorobrykh
  - Contributed by identifying the dataset and proposing the project idea. I focused on texture detection by implementing and testing HOG features, evaluating the approach using SVC and Logistic Regression models. Integrated this code into the team's shared Jupyter Notebook. In addition, I authored the Texture Detection, Support Vector Classifier, and Efficiency vs. Accuracy sections of the final report.
- Jack Corley
  - Was responsible for the color histogram features, handling the logic for feature extraction of the statistical metrics and performing a gini feature importance analysis on the extracted features and created visualizations for our analysis and report. I evaluated the performance of the three machine learning classifiers used and implemented my extracted features and code into the final three models trained. In the report I was responsible for the section of color histogram features, logistic regression, generalizability, and conclusion.
- Robert Bull
  - Technical work:
    - The main code notebook and any associated formatting, data preprocessing, EDA contributions, blob count and blob area feature, luminance histogram feature, ResNet 50 feature, all t-SNE visualizations of features, all smoothed histogram visualizations of features, implementation and execution of all classifier models (baseline, logistic regression, SVC, and random forest) with and without ResNet 50, hyperparameter tuning of classifier models, all classifier results, all confusion matrices, and all ROC curves
  - Presentation:
    - Created and presented the Blob Detection, Luminance Histogram, ResNet50, and Model Selection slides (slide numbers 9-12)
  - Paper:
    - Section 2.2 Blob Detection, 2.5 Luminance Histogram, 2.6 ResNet50, overall paper formatting, and various edits and cleanups of other sections